

Graph Compression with Edge Lists

Author One

*Department of Statistics
University of Washington
Seattle, WA 98195-4322, USA*

ONE@STAT.WASHINGTON.EDU

Author Two

*Division of Computer Science
University of California
Berkeley, CA 94720-1776, USA*

TWO@CS.BERKELEY.EDU

Editor: My editor

Abstract

Graph compression is an important problem in both theory and practice. This work considers graph compression from the perspective of edge lists.

Keywords: keyword one, keyword two, keyword three

1 Introduction

Previous graph compression algorithm considers graph compression from the perspective of node permutations. In this work, we consider the problem of sequential graph compression from the perspective of edge sequences. This work makes both theoretical and practical innovations.

2 Related Work

3 Preliminary

Let G be a graph with $V(G)$ and $E(G)$ denoting the node and edge sets, respectively. Suppose the graph has n nodes, then the node set is $V(G) = \{1, \dots, n\}$, and each edge $e \in E(G)$ is a two-element subset of V . Let $m = |E(G)|$ denote the number of edges in the graph.

A mapping $\pi : V(G) \rightarrow V(G)$ maps each node id to a new node id. If $(i, j) \in E(G) \Rightarrow (\pi(i), \pi(j)) \in E(G)$, then π is an automorphic mapping.

A graph is an abstract object with many equivalent representation forms, where each representation specifies node ids and the order of edges. Graph compression should squeeze out the extra information in representations, which can be done through the bits-back mechanism.

4 Method

4.1 Equivalent representations

In this work, we consider compressing the edge list. Edge lists from a graph using different orders are equivalent representations of the graph.

Edge permutations and orders. Let λ be a permutation defined on $\{1, \dots, m\}$: order defined on the edge set $E(G)$. To be concrete, the permutation places λ_i in the i -th location. We assume that the edge set E has an arbitrary edge order, where edges use node ids specified by V . Let $E[\lambda]$ denote the edge order using the permutation λ . There are $m!$ edge permutations and thus $m!$ edge orders in total. Let $\circ$ be a binary operation of composing edge permutations; then the set A of all edge permutations forms a permutation group.

If we directly encode an edge list in the compression, then the compressor needs to consider all possible edge lists to get bits back. An alternative choice is to group these edge orders together, then the compressor encode a group without distinguishing edge orders within a group. Then the compressor needs to squeeze out the information about the group and get bits back.

Here, we introduce two types of grouping methods by defining two types of equivalent relationships between edge orders.

Type-1 equivalence of edge orders. Let $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ be a mapping over node ids, and let $\pi[\lambda]$ denote the operation of re-labeling node ids with π .

Definition 1. Two edge orders λ and λ' are type-1 equivalent, denoted by $\lambda \sim_1 \lambda'$, if there exists an mapping π such that $\pi[\lambda] = \lambda'$. Furthermore, the mapping π is an automorphic mapping.

Theorem 1. Let $\lambda_0 = (1, \dots, m)$ be the identify permutation, then the set $\{\lambda \in A : \lambda \sim_1 \lambda_0\}$ form a subgroup of A . Furthermore, the subgroup has size $|\text{Aut}(G)|$.

Corollary 2. There are $\frac{m!}{|\text{Aut}(G)|}$ different type-1 groups to construct the unlabeled graph G .

We next find a common representation for each group. For each edge order λ , we get an edge list $E[\lambda]$. Now we convert it to an edge list R uniform for different node orders. We start from the first edge in $E[\lambda]$ and re-label the nodes to 0 and 1. Then we scan all edges in $E[\lambda]$ and assign each new node an edge with an incremental 1.

Theorem 3. $E[\lambda]$ and $E[\lambda']$ give the same R if and only if $\lambda \sim_1 \lambda'$.

Type-2 equivalence of edge orders.

Definition 2. Let G_k^λ denote the subgraph formed by $E[\lambda[1 : k]]$, then two edge orders λ and λ' are type-2 equivalent, denoted by $\lambda \sim_2 \lambda'$, if $G_k^\lambda \cong G_k^{\lambda'}$.

Theorem 4. If G is a chain graph with four nodes, then the set $\{\lambda \in A : \lambda \sim_2 \lambda_0\}$ is not a subgroup.

Theorem 5. Suppose an edge order λ gives a graph sequence $(G_1, \dots, G_m)$. Let d_k denote the number of edges in the same orbit as the last edge e_k in G_k . There are $\prod_{k=1}^m d_k$ node orders that are type-2 equivalent to λ .

We can represent all type-2 equivalent edge orders with a single representation: the orbit sequence. Let o_k be the orbit of e_k in graph G_k . All type-2 equivalent edge orders will give the same graph sequence of $G_1, \dots, G_m$ and thus the same orbit sequence $O = (o_1, \dots, o_m)$.

On the flip side, the orbit sequence uniquely decides the graph sequence and also the group of edge orders.

Corollary 6. $E[\lambda]$ and $E[\lambda']$ gives the same orbit O if and only if $\lambda \sim_2 \lambda'$.

4.2 Variational Lower Bounds of the Graph Likelihood

Now we consider the log-likelihood of a graph with the different representations. If we use the original representation,

$$\log p(G) = \log \sum_{\lambda} p(E[\lambda]) = \log \sum_R p(R) = \log \sum_O p(O) \quad (1)$$

In general, we do not use $p(G) = \sum_{\lambda} p(E[\lambda])$ to calculate the probability of G because the edge list $E[\lambda]$ is tied to a particular subgroup of automorphic node mappings. In this calculation, we will need to consider edge orders obtained from the permutations of E but exclude node orders obtained from another non-automorphic node mapping. However, excluding such edge orders is a hard problem. As a comparison, the representation with R and O eliminates the representation variance from node labelings.

With the variational inference, we can find the lower bound of the log-likelihood. For different summations, we will need to introduce $q(R|G)$ and $q(O|G)$. Then

$$\log p(G) \geq \mathbb{E}_{q(R|G)}[\log p(R) - q(R|G)] \quad (2)$$

$$\log p(G) \geq \mathbb{E}_{q(O|G)}[\log p(O) - q(O|G)] \quad (3)$$

The lower bound shows the compression rate when the bits-back model q is used with the encoding model p . Suppose we have a list of graphs to compress. Then we can first decode bits using $q(R|G)$ to decide the edge order R . Then the compressor $p(R)$ can encode these edges one by one to compress the graph.

With an edge-exchangeable model, $p(R)$ gives the same probability to R corresponding to the same G . The uniform distribution $q(R|G) = \frac{|\text{Aut}(G)|}{m!}$ creates a tight lower bound. In this case, $\log p(G) = \log p(R) - \log |\text{Aut}(G)| + \log(m!)$.

In the second case, an edge-exchangeable model does not assign the same probability to O because some O corresponds to more edge orders than others. With our calculation above, O corresponds to $\prod_{k=1}^m d_k$ permutations. The posterior distribution $q(O|G) = \frac{\prod_{k=1}^m d_k}{m!}$ makes the lower bound tight. The log-likelihood $\log p(O) - \sum_{k=1}^m \log d_k + \log(m!)$.

Note that our lower bound does not require the model to be exchangeable. However, the bits-back encoding is a deterministic procedure. To understand the compression rate from this lower bound, we consider the decoding of R by $q(R|G)$ from compressed bits as a random sample of R from the distribution $q(R|G)$. In this case, the compressed rate can be considered a random realization of the expectation in the lower bound. Therefore, when we compress a large number of graphs, the compression rate will converge to the lower bound.

4.3 Sequentialize the Lower Bound

When we compress a large graph, we need to sequentialize the lower bounds so that the compression and bits back can work for one edge after another. In particular, we need to establish a recursive procedure to compress the graph.

We first work with orbits given by type-2 equivalent edge orders. Note that the orbit order corresponds to a subgraph sequence $(G_1, \dots, G_m)$. We can construct the graph G with the orbit sequence with $p(o_k|G_{k-1})$. The information of the orbit sequence can get some bits back to improve the compression rate. We will need an orbit model $q(o_k|G_k)$ to get bits back.

$$\log p(G_k) = \log \sum_{o_k} p(G_{k-1})p(o_k|G_{k-1}) \geq \mathbb{E}_{q(o_k|G_k)}[\log p(G_{k-1}) + \log p(o_k|G_{k-1}) - \log q(o_k|G_k)] \quad (4)$$

This inequality shows the recursive relationship. From the encoding perspective, first choose the orbit o_k by decoding bits from the compressed orbits (with $k+1$ and later ones), and then encode it with $p(o_k|G_{k+1})$. The compression step saves $-\log q(o_k|G_k)$ bits and spends $-\log p(o_k|G_{k-1})$ bits. In typical cases, there are more choices for o_k when adding an edge to G_{k-1} than for choosing an orbit in the graph G_k , though there are counterexamples. Overall, the compressor needs $\sum_{k=1}^m (-\log p(o_k|G_{k-1}) + \log q(o_k|G_k))$ bits to encode the graph through an orbit sequence O .

In the sequential case, this lower bounds also holds for non-exchangable model $p(o_k|G_{k-1})$. **Sequentializing the probability of R .** Sequentializing R is substantially more difficult than sequentializing O . The difficulty comes from the fact that edges in R are entangled: a later edge refers to node ids used by any prior edges. Instead of using R , we consider an alternative representation with partially labeled subgraphs and their orbits.

Let $\bar{G}_{m-k}$ be a partially labeled subgraph with $(m-k)$ edges. All nodes that appear in the first k edges are labeled with their ids. Then $(R[1:k], \bar{G}_{m-k})$ will complete the full graph G . We consider the two ends of the subgraph sequence: $\bar{G}_0$ is an empty graph, and $\bar{G}_m$ is the unlabeled graph G .

We show that the subgraph sequence $(\bar{G}_{k'} : k' = 0, \dots, m)$ is an equivalent representation of R . It is clear to see that R decides each subgraph $\bar{G}_{k'}$. We can also show that $\bar{G}_{k'}$ and $\bar{G}_{k'+1}$ decides the edge $R[m-k']$.

Theorem: if there are two sequences R_1 and R_2 such that $R_1[1:k-1] = R_2[1:k-1]$ but $R_1[k] \neq R_2[k]$. Then $R_1[k]$ and $R_2[k]$ are not in the same orbit in $\bar{G}_{m-(k-1)}$.

Proof: Note that the orbit of the subgraph are computed from known node ids and connections, so orbits contain all information about edge labels. If a node with an id is contained in $R_1[k]$ but not in $R_2[k]$, then they must different orbits. From $R_1[k-1]$ and $R_2[k-1]$, we have the same subgraph $\bar{G}_{m-(k-1)}$. If $R_1[k] \neq R_2[k]$ have different numbers of unlabeled nodes in $\bar{G}_{m-(k-1)}$, then they clearly are in different orbits decided by $\bar{G}_{m-(k-1)}$. Now we consider the three cases where $R_1[k]$ and $R_2[k]$ each have 0, 1, or 2 labeled nodes. First, it is not possible that they each have 0 labeled nodes; otherwise $R_1[k]$ and $R_2[k]$ are both isolated edges, and $R_1[k] \neq R_2[k]$ cannot be true. If both $R_1[k]$ and $R_2[k]$ have 1 unlabeled node, then this node will get the same label. They must have different labeled nodes, so they are in different orbits. In the third case, the two nodes of $R_k[k]$ have labels, then $R_1[k] \neq R_2[k]$ cannot have the same orbit.

Corollary: $(\bar{G}_m, \dots, \bar{G}_{m-k})$ is an equivalent representation of $R[1:k]$.

Now we work with the subgraph sequence $(\bar{G}_{k'} : k' = 1 : m)$, which is equivalent to compressing edges in R in the backward direction. The key is to define the two distributions

$p(\bar{G}_{k'}|\bar{G}_{k'-1})$ and $q(\bar{G}_{k'-1}|\bar{G}_{k'})$.

$$\log p(\bar{G}_{k'}) = \sum_{\bar{G}_{k'-1}} p(\bar{G}_{k'-1}) p(\bar{G}_{k'}|\bar{G}_{k'-1}) \quad (5)$$

$$= \mathbb{E}_{q(\bar{G}_{k'-1}|\bar{G}_{k'})} [\log p(\bar{G}_{k'-1}) + \log p(\bar{G}_{k'}|\bar{G}_{k'-1}) - q(\bar{G}_{k'-1}|\bar{G}_{k'})] \quad (6)$$

This variational lower bound shows the compression rate.

We first construct $q(\bar{G}_{k'-1}|\bar{G}_{k'})$. From the perspective of completing the edge list R , $\bar{G}_{k'}$ contains nodes referred by the first $(m - k')$ edges and also the last $(n - k')$ edges in R . The task for q is to identify which is the next edge in R . The sample space for q is

- an edge incident with two nodes with ids
- an edge incident on one node with an id and one node without a node id
- an edge incident with two nodes without ids

q samples one orbit to identify one of these edges. Therefore, q gets bits back by decoding the orbit.

After the edge is chosen, we derive $\bar{G}_{k'-1}$. For each one of the two new nodes in this edge, we first assign a new id to it if it does not have one. Then we remove the edge. If a node has all labeled neighbors, the node is also removed from the $\bar{G}_{k'}$.

Now we consider the distribution $p(\bar{G}_{k'}|\bar{G}_{k'-1})$. The compressor should compress the new edge (i, j) using their ids. If we could identify restrictions about the values of i and j , then we can improve the compression rate. The compressor also needs to indicate which nodes will lose their id in $\bar{G}_{k'}$. One the decompression end, we assume the decompressor already has $\bar{G}_{k'-1}$. It first decompresses to get (i, j) and then two indicators, whether to remove the node ids. After that, it gets $\bar{G}_{k'}$. Then it compresses the orbit of (i, j) in $\bar{G}_{k'}$ to give bits back to the decompression stream.

We implement the algorithm with the following three procedures. Note that we use a negative node value to indicate that a node is unlabeled.

5 Experiments

Algorithm 1 Update completing graph

```

1: Input: Graph  $\bar{G} = (V, E)$ , edge  $(i, j)$ , graph size  $n$ 
2: Output: Graph  $\bar{G} = (V, E)$ , edge  $(i, j)$  with new ids.
3: Count the number of unlabeled nodes  $r = \sum_{i \in V} 1[i < 0]$ 
4:  $b_i \leftarrow 0, b_j \leftarrow 0$ 
5: if  $i < 0$  then
6:    $b_i \leftarrow 1$ 
7:   Update the node id  $i \leftarrow (n - r + 1)$  in the subgraph  $\bar{G}$ 
8:    $r \leftarrow r - 1$ 
9: end if
10: if  $j < 0$  then
11:    $b_j \leftarrow 1$ 
12:   Update the node id  $j \leftarrow (n - r + 1)$  in the subgraph  $\bar{G}$ 
13:    $r \leftarrow r - 1$ 
14: end if
15: Remove  $(i, j)$  from  $\bar{G}$ 
16: Remove  $i$  from  $\bar{G}$  if  $i$  has degree 0
17: Remove  $j$  from  $\bar{G}$  if  $j$  has degree 0
18: return  $\bar{G}, (i, j), (b_i, b_j)$ 

```

Algorithm 2 The compression algorithm

```

1: Input: Graph  $G = (V, E)$ , initial compressor state  $s_0$ ,
2: Output: final compressor state  $s_{m+1}$ 
3:  $\bar{G}_0 \leftarrow (V, E)$  with all node indices negated
4: for  $k = 1, \dots, m$  do
5:    $o_k, s'_k = \text{q.decode}(s_{k-1})$ 
6:   Identify an edge  $(i, j)$  from  $\bar{G}_{k-1}$ 
7:    $\bar{G}_k, (i, j), (b_i, b_j) = \text{Update}(\bar{G}_{k-1}, (i, j), |V|)$ 
8:    $s_k = \text{p.encode}(s'_k, (i, j), (b_i, b_j))$ 
9: end for
10:  $s_{m+1} = \text{p.encode}(s_m, (|V|, |E|))$  ▷ Need to encode the size of the graph
11: return  $s_{m+1}$ 

```

Algorithm 3 The decompression algorithm

```

1: Input: Initial compressor state  $s_{m+1}$ ,
2: Output: graph  $G$ 
3: Set  $G_m$  to be empty graph
4: Set  $R = []$ 
5:  $(n, m), s_m = \text{p.decode}(s_{m+1})$ 
6: for  $k = m, \dots, 1$  do
7:    $(i, j), (b_i, b_j), s'_k = \text{p.decode}(s_k)$ 
8:    $R[m] = (i, j)$ 
9:    $\bar{G}_{k-1} \leftarrow \bar{G}_k \cup (i, j)$ 
10:  if  $b_i == 1$  then
11:    Update the node id  $i \leftarrow -i$  in  $\bar{G}_{k-1}$ 
12:  end if
13:  if  $b_j == 1$  then
14:    Update the node id  $j \leftarrow -j$  in  $\bar{G}_{k-1}$ 
15:  end if
16:  Let orbit  $o_k$  be the orbit of  $(i, j)$  in  $\bar{G}_{k-1}$ 
17:   $s_{k-1} = \text{q.encode}(s'_k, o_k)$ 
18: end for ▷ All node ids in  $\bar{G}_0$  should be negative (unlabeled)
19: return  $R$ 

```

Appendix A.

Add proofs here

References